Proof-Gated Live Updates: Admitting a Code Change into a Running Native Program Only With a Certificate It Re-Checks Itself

Tunjay Akbarli (Omnira CJSC) -- author list and affiliations to be confirmed
Draft of 2026-10-15. Results marked TODO have not been measured yet.

Abstract

Production services are patched under time pressure, and increasingly with changes proposed by automated tools. Dynamic software updating (DSU) lets a running process take a new version without restarting, but the systems that exist either trust the patch or prove only that it is type safe. Offline tools can prove two versions equivalent, but they are not part of the decision a running process makes about whether to accept a change. We present *proof-gated live updates*, implemented in Codira, an ahead-of-time compiled, garbage-collected language whose compiler already proves contracts and effect annotations with an SMT solver. An update is a new version, a declaration of intent, and a certificate. The certificate shows three things: every function the patch did not mean to change still computes the same results and performs the same effects, every changed function still meets its published contract and effect guarantees, and the live heap migrates to the new layouts without losing or reinterpreting data. The running process re-checks the certificate with an SMT solver alone, so it does not have to trust whoever produced the update. It then parks every task at a safe point at which no affected function is active, snapshots the heap, swaps the code through a dispatch table, and restores the snapshot if the new version faults. We describe the definition of a safe update and its soundness argument, a relational verifier that handles straight-line code, calls, loops, memory and effects in increasing levels, and the runtime protocol. **[TODO: evaluation summary, written last: number of updates in the corpus, fraction admitted automatically by level, proof and re-check times, pause times, steady-state overhead, and how many faulty updates were rejected or rolled back and how many were admitted and wrong.]**

1 Introduction

A service that has to stay up is patched while it runs. The patch is written in a hurry, by someone who did not write the code it touches, and, more and more often, by a tool: a fuzzer-driven repair system or a language model proposes a change to a vulnerability, and an operator has to decide whether to deploy it. Restarting is expensive for stateful services, so dynamic software updating (DSU) systems let the running process take the new version in place [1, 2, 3, 4, 5]. What those systems check before they apply a change is little. Ginseng and Proteus analyse that an update is type safe and representation consistent [2, 6]; the first system to make patches verifiable native code did so to establish type safety [1]; Linux livepatch checks that no task is inside a function being replaced [7]. None of them checks that the patched program still *means* what the old one did where the change was not meant to reach, or that it still meets the properties a service promises its callers.

The consequences are concrete. A patch can change the behaviour of a function nobody mentioned, because a shared helper moved. It can weaken a postcondition that callers rely on. It can add an allocation to a path that was promised never to allocate. It can migrate a long-lived data structure through a conversion that silently truncates. And an unchecked hot update applies all of these to a live heap that cannot be rebuilt from scratch.

Verification of such differences exists, but it is run *beside* the update rather than *inside* the decision to apply it. Regression verification proves two versions equivalent [8]; SymDiff and the merged-program approach of Hayden et al. check differential properties of an update, offline and with user-written specifications [9, 10]; VeriBin checks

functional equivalence of binary patches before a vendor deploys them [11]. The running process takes no part: it receives a new library and trusts that someone ran the tool.

Insight. Suppose the language has three properties. First, every call between functions already goes through a patchable table, because hot reload needs it, so an update is a small, well-defined edit of one data structure. Second, every heap object carries runtime type information, so a layout change can be migrated mechanically. Third, the compiler already proves, with an SMT solver, the contracts and effect annotations that describe what each function promises. Then the property "this update preserves what it should" can be stated in the language's own terms, proved with the machinery the compiler already has, and handed to the running process as a *certificate*, in the style of proof-carrying code [12]: the process re-checks the certificate with the solver and nothing else, and decides.

What we claim. We do not claim to be the first to verify a software update; Section 11 lists the closest work and what it does not do. We claim a particular combination, built into the admission path of a running native runtime:

- a definition of a *safe update* that joins behavioural equivalence outside a declared intent, contract and effect preservation, and heap migration, together with an *affected set* of functions that must not be active when the swap happens (Section 4);
- a certificate that a process can re-check with an SMT solver alone, and an explicit account of what that does and does not trust (Section 3 and Section 5);
- a relational verifier, built on the contract verifier of a real compiler, in levels from straight-line integer code to loops, memory and effect traces (Section 6);
- a runtime protocol with safe points, a snapshot, migration, a dispatch-table swap and automatic rollback, with a test for each failure mode (Section 7);
- **[TODO: an evaluation on update histories of three services, with fault injection, once the corpus and harness exist (the Evaluation section).]**

What the certificate does not establish. It does not show that the *intent* is the right one: if the author declares a function changed and replaces its contract with a weaker one, the certificate faithfully proves the weaker thing. It does not remove the compiler from the trusted base: a certificate is about source semantics and about the bytes of a library by hash, and the compiler that turned source into those bytes is trusted except where translation validation covers a transformation (Section 3). What it removes from the trusted base is the *producer of the update*.

2 Background: the parts of Codira an update touches

Codira is an ahead-of-time compiled, statically typed, garbage-collected language. We describe only what the rest of the paper uses.

Dispatch table and hot reload. A compiled module is a shared library (a `.codiralib`) that exports a small fixed set of entry points. Every call from one Codira function to another is an indirect call through a table of function pointers in the library's data section; none is a direct call to an address. A call therefore depends on the callee's *slot* and signature, not on its address or body. Hot reload links a rebuilt library and repoints the table.

Typed heap. Every heap object has a type descriptor with a stable identifier and the offsets of its fields, including which fields hold references. The collector and the migration code use the same descriptors. When a struct's layout changes between versions, migration maps the old object to the new layout by field name, converts retyped numeric fields with defined saturating semantics, and keeps a renamed field's value when the new version marks it with `@renamed(from: "old")`. The whole heap can also be written to a byte stream and restored in a runtime running the same program (a *heap snapshot*), refusing a snapshot whose layouts do not match.

Contracts, refinement types and effects. Functions may carry `requires` and `ensures` clauses, loops may carry `invariant` and `decreases`, and types may carry refinement predicates. The compiler proves them statically with an SMT solver using exact bit-vector semantics for each integer width, so a wrapping overflow is visible to the proof, and it reports a counterexample when a proof fails. Functions may be annotated `@noalloc`, `@nopanic`, `@noblock` or `@realtime`; the compiler computes, by a fixed point over the call graph, which effects each function can have, and reports the chain of calls behind a violation. Integer division counts as a possible panic unless the solver proves its divisor non-zero.

Fault interception. A separate runtime component (HERACLES) intercepts hardware faults and fuel exhaustion in running Codira code and can run a recovery strategy instead of terminating the process.

Closures and generics. Function values are runtime-owned code cells that are repointed on every reload, and each instantiation of a generic function is compiled separately. Both matter for the *affected set* (Section 4), because a closure body and a generic instance are functions in their own right.

3 Threat model and trusted base

Who is untrusted. The producer of the update: a human who made a mistake, a tool that proposes patches, or an attacker who can submit a candidate update to the process. The producer controls the new source, the new library bytes, the intent file and the certificate.

What the process trusts.

- The SMT solver (Z3 [13]) decides each obligation correctly. A solver bug can admit a bad update.
- The *verification-condition generator*: the code that turns source into SMT queries. If it models the language incorrectly, a certificate can prove a statement that is not true of the program. In this work the generator is the same symbolic executor the compiler uses to prove contracts, so a defect there already affects ordinary builds.
- The *compiler's translation of source to machine code*. A certificate is about source semantics; it binds queries to the bytes of the library by cryptographic hash, but it does not prove that those bytes implement the source. Translation validation covers some Eidos optimizer transformations [14, 15, 16] and the LLVM back end is trusted. This differs from proof-carrying code [12], whose goal is to remove the code producer, compiler included, from the trusted base for a *safety policy* checked on machine code. Our goal is narrower: remove the producer of the *update*.
- The hash function that binds queries to code bytes, and the runtime itself.

What is out of scope. Whether the declared intent is the right one; stack overflow (no stack bound is computed); calls into foreign C code; shifts by the operand width or more, which are undefined in the compiled code; and an update whose old and new libraries disagree with the compiler version that produced the certificate (the certificate records it, and the checker rejects a mismatch).

4 Safe updates

We state what the checker establishes. Let a *version* $V = (F, T, C)$ consist of a set of functions F with bodies, a set of types T with layouts, and the contracts and effect annotations C of its functions.

> **Definition 1 (intent).** An intent I for an update from V to V' names a set $\Delta \subseteq F$ of *changed* functions, and for each $f \in \Delta$ one relation: *equal under* φ (the two bodies agree on every input satisfying the precondition φ), *refines* (the new body meets the old contract), or *replaces* (the new body meets a new

contract C'_f stated in the intent). It also names a set $\Theta \subseteq T$ of *retyped* types, each with a migration μ_τ: the automatic field mapping or a user-written `migrate` block.

Definition 2 (affected set). The affected set $A(V, V', I)$ is the smallest set of functions of V that contains Δ, contains every function whose body reads or writes an object of a type in Θ or holds a value of such a type in a local, and contains every function that is, or whose closure bodies and generic instances are, a member of the set above.

Definition 3 (safe update). The update is *safe* when all of the following hold. (E) For every $f \in F \setminus \Delta$ present in both versions, f and f' compute the same result, perform the same effect operations in the same order, and leave the same fields and array elements in the same state, given that every callee of f either is unchanged or meets its contract in V'. (C) For every $f \in \Delta$, the stated relation holds, and the contract of f' is no stronger in its `requires` and no weaker in its `ensures` than that of f unless the intent says *replaces*. (F) For every function annotated `@noalloc`, `@nopanic`, `@noblock` or `@realtime` in V, the same effect summary excludes those effects in V'. (M) For every $\tau \in \Theta$, the migration μ_τ is total on every value that can occur in a live object of τ, or the values on which it is not total are rejected before the swap.

The affected set and active frames. The swap changes code and the layout of heap data at one instant. A task whose stack holds a frame of a function in A would resume on old code against migrated data, or on old code that is no longer the code the table points to, and could read a field at the wrong offset. DSU systems face the same hazard and answer it with explicit update points [3] or a static analysis showing that no active frame is affected [6]. We require:

Invariant 1. At the instant of the swap, no frame of any task belongs to a function in A.

The runtime enforces Invariant 1 (Section 7); the certificate states A and the checker verifies that every function outside A has an obligation of kind (E). A function missing from A that in fact touches a migrated type would make the update unsound, which is why Definition 2 closes A under closures and generic instances and why the prover tests that case (Section 9).

Theorem 1 (soundness, conditional). Assume the solver is sound, the verification-condition generator agrees with the language's semantics, the hash binds the library the process loads, and Invariant 1 holds at the swap. If the checker accepts an update, then (E), (C), (F) and (M) hold of that update, and from the swap on every call to a function in $F \setminus A$ behaves as before, *partially*: wherever both versions terminate.

Proof sketch. The checker accepts only when every obligation is unsatisfiable under the recorded resource limit; each obligation is the negation of one conjunct of Definition 3 for one function or type. For (E), take $f \notin \Delta$ and argue by induction on the depth of the call graph below f, treating a recursive call as the induction hypothesis, which is the standard rule of regression verification [8]. At depth zero the obligation compares the two bodies directly. At depth k each callee g is either unchanged, in which case it is the same uninterpreted function on both sides and the induction hypothesis says it computes the same value, or changed, in which case the obligation holds for any function meeting its contract. Termination parity is shown only where both bodies carry a `decreases` clause that the contract verifier proves; otherwise the result is *partial* equivalence. (C) and (F) are the contract implication and effect-summary comparison of Section 5. (M) is the migration totality obligation. Invariant 1 ensures no frame straddles the swap, so the statements about complete calls are the statements about every call after the swap. □

The theorem is a statement about the *checker's acceptance* given the stated assumptions; it is not a verified implementation. Section 9 reports how often it is exercised by faulty updates.

A worked example. A token-bucket limiter refills its balance by `elapsed * rate`, capped at `cap`.

```
public func refill(tokens: i64, elapsed: i64, rate: i64, cap: i64) -> i64
    requires 0 <= tokens && tokens <= cap && 0 <= elapsed && 0 <= rate && 0 < cap
    ensures 0 <= result && result <= cap
{
    let sum = tokens + elapsed * rate;
    if sum > cap { cap } else { sum }
}
```

The compiler's own verifier models wrapping arithmetic exactly, so it refuses this version: when `elapsed * rate` overflows, `sum` can be negative and the postcondition fails. The fix avoids the multiplication when it would exceed the headroom:

```
    if rate == 0 { tokens }
    else if elapsed > (cap - tokens) / rate { cap }
    else { tokens + elapsed * rate }
```

Under the precondition that the old body does not overflow, the two versions agree: for `rate > 0`, `elapsed > (cap - tokens) / rate` holds exactly when `elapsed * rate > cap - tokens`, which is the old test. The intent therefore says: `refill` is *equal under* "no overflow in the old body" and *refines* the old contract (the new body also proves the postcondition for all inputs). Consider three candidate patches. A cosmetic change from `>` to `>=` in the old test is admitted, because the two return the same value at equality. A patch that replaces the new test by `elapsed >= (cap - tokens) / rate` is rejected with a counterexample: with `tokens = 0, cap = 5, rate = 2, elapsed = 2`, the old body returns 4 and the patch returns 5. An unchecked hot update would apply that patch to a live limiter. [NOTE: this example was derived by hand; replace the counterexample with the one the prover prints, and add the corpus identifiers of the three patches.]

5 Certificates and the checker

An update certificate is a self-contained list of obligations plus the facts that bind them to the update.

```
certificate := header, affected-set, obligation*
header      := old_hash, new_hash, intent_hash, solver_id, rlimit, compiler_id
obligation  := kind (equivalence | contract | effect | migration),
               subject (function or type symbol),
               query   (SMT-LIB2), expected (unsat)
```

The certificate builds on the one `codira certify` already writes for a single build: SMT-LIB2 queries with the hashes of the code ranges they describe, re-checked without the compiler's front end. An update certificate adds the old-to-new relation, the intent hash and the affected set.

Checking. The checker (i) verifies the hashes of the old and new libraries and of the intent; (ii) rejects a solver identifier or compiler identifier it does not accept; (iii) checks that the affected set is closed as in Definition 2 over the call graph recorded in the new library and that every function outside it has an obligation of kind equivalence; (iv) runs every query under the recorded deterministic resource limit and accepts only `unsat`. A query that times out, returns `sat`, returns `unknown`, or fails to parse rejects the update. Nothing is assumed on a timeout.

Contract and effect obligations. Contract changes are decided by Z3 as implications between the old and new clauses over the parameters, as in the `codira api-diff` tool: a `requires` that became stronger and an `ensures` that

became weaker are breaking unless the intent says *replaces*. Effect obligations compare the effect summaries of Section 2 before and after.

Migration obligations. For each retyped type the checker compares the layouts, derives the field mapping, and for each retyped field proves that the conversion is total on the values the type's refinement predicate allows, or reports the failing value class.

6 Relational verification

The checker re-checks obligations; the *prover* produces them. Its core is a relational verifier: given a function f and its new version f', it proves that they agree. We built it on the symbolic executor the compiler uses to prove contracts, because that executor already models integers exactly at every width, handles loops with invariants and measures, and handles `for` loops over ranges and arrays, calls, and contracts. Translation validation of the optimizer (`codira_tv`) covers only loop-free, call-free integer code over unbounded integers, with an explicit overflow caveat, so it is not the foundation here; we use bit-vectors instead.

The prover works in levels. Each level is a usable result, and the evaluation reports, per update, the highest level it needed.

- **L1: straight-line and branching integer code.** Both bodies are executed symbolically from the same fresh inputs, and the prover asks the solver whether the results can differ.
- **L2: calls.** Each callee unchanged by the update is replaced by the *same* uninterpreted function on both sides, so a change to one function is proved in isolation. A changed callee is replaced by its contract in the new version.
- **L3: loops.** Two loops are paired when their structure matches, and the prover proves a coupling invariant inductively. The default invariant is equality of every live variable at the loop head, which is the automatic invariant that makes regression verification work in most practical cases [8]; user `invariant` clauses are used when given. Loops over ranges and arrays are paired by their bounds. A loop pair the prover cannot couple is reported as unsupported, naming the construct.
- **L4: memory.** Fields and array elements are modelled in the SMT theory of arrays. The intent lists the fields allowed to differ; every other field must be equal in the two final states.
- **L5: effect traces.** The sequence of effect operations (channel sends and receives, allocations, panics) is compared, or the declared difference is.

Anything the levels do not model is `Unsupported`, with a reason, and the update is rejected unless the intent marks the function *replaces* with a new contract that is proved on its own. A function the verifier cannot encode is never silently accepted.

Related techniques. The structure follows regression verification [8] and relational Hoare logic and product programs [17, 18], and shares its goal with SymDiff [9]. The difference is where it runs and what it is for: it is the first half of an admission decision, and its output is re-checkable by a process that has no part of the prover.

7 The runtime protocol

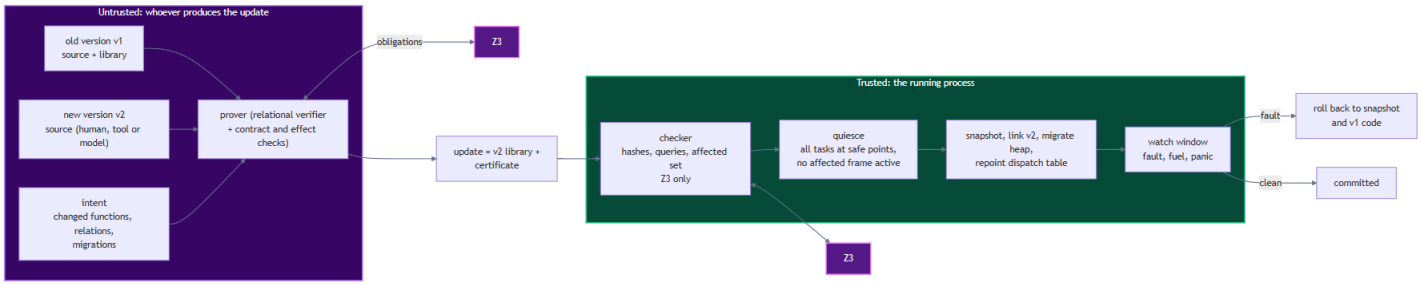


Figure 1: The update pipeline. The producer is untrusted; the process re-checks the certificate with Z3 alone and enforces the swap protocol.

An update moves through the states of Figure 2.

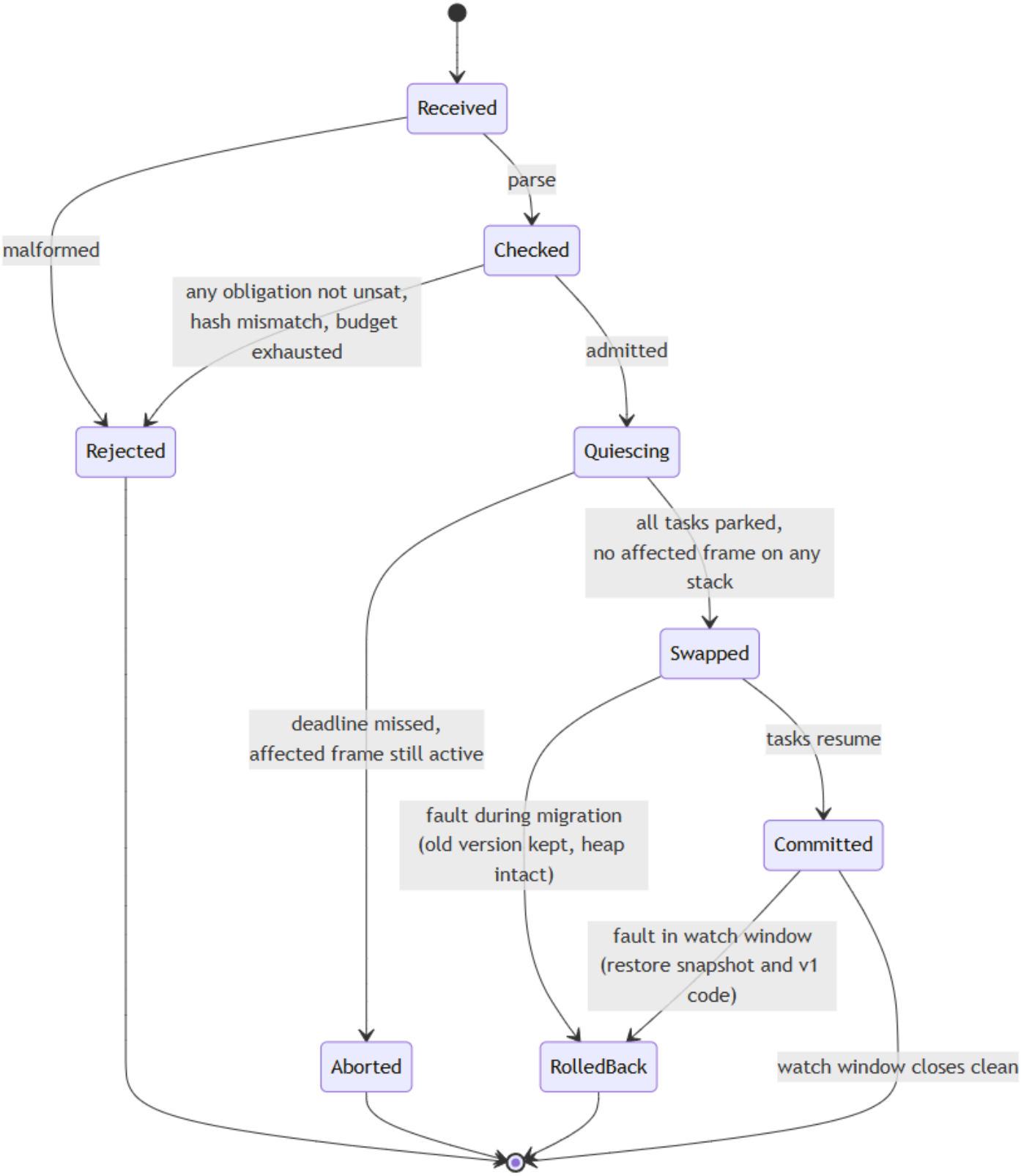


Figure 2: The states of an update in a running process. Every path that does not reach Committed leaves the process on the old version with an intact heap.

1. **Received and Checked.** The process parses the update and runs the checker of Section 5. Failure is Rejected.
2. **Quiescing.** Every task is brought to a *safe point*: the worker pool's tasks, tasks blocked on channels or in a nested scope (a blocked task is parked, which counts), and host threads inside calls into the runtime. The runtime has no stack maps, so collection and snapshotting are only valid while no other thread runs Codira

code; safe points exist to guarantee exactly that. Quiescing has a deadline; missing it is Aborted, with no state changed.

3. **The active-frame rule.** Quiescing does not complete while any task has a frame of a function in the affected set on its stack (Invariant 1). A long-running task that stays inside an affected function simply prevents the update until it leaves, and the update is Aborted at the deadline. **[TODO: describe the mechanism the implementation uses to know which functions are active (a per-task shadow stack of function identifiers in builds that ask for updatability, or poll-site information), its cost, and how often realistic updates are blocked by it.]**
4. **Swapped.** The runtime takes a heap snapshot, links the new library, migrates the heap, and repoints the dispatch table. A failure before the table is repointed leaves the old state: linking happens before migration, so a failed link never leaves a migrated heap under old code. A fault during migration restores the snapshot.
5. **Committed and the watch window.** Tasks resume. For a configured window a fault intercepted by the fault-handling component, a fuel exhaustion, or a panic from the new code restores the snapshot and the old code and the program continues on the old version. Old code stays mapped until the window closes.
6. **Serialization.** Updates are serialized; a second update is checked against the result of the first.

[NOTE: states 2 to 6 describe the protocol as specified in RFC 3; as of this draft the repository has the pieces (dispatch-table relink, a failed link leaving the heap alone, heap snapshots, fault interception) but not the quiescence protocol, the rollback path or the admission gate. the Implementation section records what exists at each commit.]

`migrate` blocks. When a type's layout changes in a way the automatic field mapping cannot express, the author writes a `migrate T v1 -> v2 { ... }` block. The prover treats its result as a value that must satisfy the new type's invariants, and the checker's (M) obligation covers it. **[TODO: syntax, restrictions, and what is proved.]**

8 Implementation

The system is built inside the Codira compiler and runtime, a Rust workspace. Table 1 gives the size of the pieces it builds on at the commit of this draft, counting non-test Rust lines.

Table 1: Existing components the update system builds on, at commit `5e18c5eae8` (non-test Rust lines).

Component	Role in this work	Lines
`codira_hir::verify`	contract verifier and symbolic executor	6,963
`codira_hir::realtime`	effect summaries (`@noalloc` and others)	1,163
`codira_tv`	translation validation of optimizer rewrites	1,400
`codira_smt`	Z3 bindings, deterministic resource limit	1,644
`codira_certificate`	build certificates and their checker	797
`codira_runtime`	loading, relinking, tasks, host interface	4,154
`codira_memory`	collector, migration, heap snapshots	8,537
`codira_healing`	fault interception (HERACLES)	4,452
`codira_abi`	type descriptors, dispatch table layout	2,279

Table 2 records, for each part of the design, whether it exists in the repository.

Table 2: Status of each part of the design at the time of writing.

Part	Status

Part	Status
Dispatch-table relink; a failed link leaves the heap alone	present
Field-name heap migration, `@renamed`, saturating retyping	present (commit `d09028e331`)
Heap snapshot and restore	present (commit `d09028e331`)
Contracts, bit-exact, loops, `for`, `continue`	present (commits `f71a80ef7d`, `1a7eef3a3f`)
Effect summaries	present (commit `a5919f822c`)
Build certificates and `codira certify`; `codira api-diff`	present (feat/tooling, merged to `main`)
Relational prover L1 to L5, update certificate, affected set	**[TODO: status at submission]**
Quiescence, `apply_update`, rollback, active-frame rule	**[TODO: status at submission]**
`migrate` blocks	**[TODO: status at submission]**

9 Evaluation

[TODO: this whole section is a plan until the harness runs. Nothing below is a measurement.]

Questions.

1. *Coverage*. What fraction of realistic updates is admitted automatically, by prover level? Which are rejected, and are the rejections correct?
2. *Cost*. What are proof time, certificate size and re-check time?
3. *Disruption*. What is the pause at the swap, as a function of heap size and task count, and how does it compare with unverified hot reload?
4. *Steady state*. What does the dispatch-table indirection, plus any safe-point polls, cost compared with plain ahead-of-time calls?
5. *Safety*. Given faulty updates (hand-written, mutation-generated, model-proposed), how many are admitted, rejected, or caught by rollback, and how many are admitted *and wrong*? Zero admitted-and-wrong is the target, and any case is reported.

Setup. **[TODO: confirm and fill in.]** All measurements run on one machine: Intel Core i9-14900HX, 32 logical cores, 64 GB RAM, Windows 11 Home, under constant background load from other builds. Because of that load, every timing is reported as a minimum and a median over at least 25 interleaved runs, with the spread, and the load condition is stated.

Workloads. Three long-running services written in Codira for this evaluation, driven by a host harness through the runtime's call interface: a key-value store with an open-addressing hash table and a write-ahead log, a ledger and rate limiter with balance invariants stated as contracts, and a stream aggregator with worker tasks and channels. They are small programs written for this study, not production systems. The language has no networking or string library yet, so the workloads use integers, structs, enums, optionals, closures, generics, arrays and channels, and the services are exercised through function calls, not sockets. **[TODO: sizes in lines and functions of each workload.]**

Update histories. For each workload an authored history of at least twenty updates, labelled by category and provenance: bug fixes, behaviour-preserving refactors, performance changes, features that change layouts (some with a `migrate` block), seeded regressions that look plausible, contract changes in both directions, and effect changes. **[TODO: the mix actually used, and how each update was derived (adapted from a change pattern in a public project, or invented), because a reader will ask.]**

Baselines. (a) Unverified hot reload, the behaviour of the runtime today. (b) Restart and reload from a snapshot. (c) The same prover run offline after the fact, to separate the value of gating at load time from the value of proving at all.

Fault injection. A mutation generator over the workload source (flip a comparison, off-by-one in a bound, change a constant, delete a bounds check or a contract clause, swap operands, add an allocation to a hot path, change a field type) produces faulty updates from the real histories. For each we record whether the baseline admits it, the gate rejects it, rollback catches it, or it is admitted and wrong.

Table 3: Coverage of the prover on the update corpus, by the highest level needed.

Level	Updates	Admitted	Rejected (correct)	Unsupported
L1 straight-line	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
L2 calls	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
L3 loops	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
L4 memory	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
L5 effects	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**

Table 4: Cost of admission per update (minimum / median over 25 or more interleaved runs).

Workload	Proof time	Certificate size	Re-check time
Key-value store	**[TODO:]**	**[TODO:]**	**[TODO:]**
Ledger	**[TODO:]**	**[TODO:]**	**[TODO:]**
Aggregator	**[TODO:]**	**[TODO:]**	**[TODO:]**

Table 5: Pause at the swap by live heap size and task count.

Heap	1 task	8 tasks	64 tasks
1 MB	**[TODO:]**	**[TODO:]**	**[TODO:]**
64 MB	**[TODO:]**	**[TODO:]**	**[TODO:]**
1 GB	**[TODO:]**	**[TODO:]**	**[TODO:]**

Table 6: Outcomes of fault injection by approach.

Outcome	Unverified reload	Gate only	Gate and rollback
Rejected before the swap	n/a	**[TODO:]**	**[TODO:]**
Admitted and caught by rollback	n/a	n/a	**[TODO:]**
Admitted and wrong	**[TODO:]**	**[TODO:]**	**[TODO:]**

Threats to validity. The corpus is authored by us, not collected from deployed systems; the workloads are small; the machine is loaded and a single one; the solver's deterministic resource limit makes results reproducible on one build but they can differ across solver versions; and the semantic gap between source and machine code is closed only where translation validation applies. **[TODO: add whatever the evaluation turns up.]**

10 Discussion and limitations

The intent is trusted. A wrong intent proves the wrong thing. The checker can only refuse changes the intent does not cover; it cannot know what the operator wanted. For automatically proposed patches the intent can be

derived from the diff, which puts the proposer's own diff in the trusted base, or *written by a person*; the right default for untrusted proposers is the second.

Partial equivalence. Without termination measures on both sides, equivalence holds only where both terminate. A patch that turns a terminating function into a looping one is caught only where a `decreases` clause exists.

Active frames can block an update. A task that never leaves an affected function prevents the swap. Kitsune's answer is to let the programmer name update points in the main loop [3]; ours is to refuse and abort. Section 9 measures how often realistic updates hit this.

Native code the process cannot see. An update that changes a foreign C library the program links cannot be proved by this system and is rejected unless the intent declares it trusted.

Stack overflow and undefined shifts are outside the guarantees, as in the compiler's other analyses.

Generality. The design depends on a dispatch table, runtime type information for every heap object, and a contract and effect language that a solver understands. It is a result about a language designed for it, not a technique for patching arbitrary C programs.

11 Related work

Dynamic software updating. Hicks, Moore and Nettles introduced patches that carry both new code and state-transition code, expressed as verifiable native code, to establish type safety [1]. Proteus gives a formal calculus with provable type safety and representation consistency, and an updateability analysis that can prove an update independent of the program's state [6]. Ginseng applies this to C with automatically generated patches [2], Kitsune updates a whole program at developer-named update points with generated state transformers [3], and Rubah does so for Java on a stock JVM [5]. Ksplice and Linux livepatch update kernels, with a consistency model that decides when each task may switch [4, 7]. Erlang provides hot code loading as a language feature [19]; a survey covers the field [20]. These systems establish type safety, state consistency or safe switching times. They do not prove that the new version preserves the old version's behaviour where the change was not meant to reach, and none re-checks a proof at load time.

Verifying updates. Hayden et al. give the first methodology we know of for automatically verifying the correctness of dynamic updates: a merged program combining old and new versions, user-written client-oriented specifications, and off-the-shelf verifiers, evaluated on updates to Redis and synthetic programs [10]. Regression verification proves equivalence of similar programs with callees as uninterpreted functions and automatic loop invariants [8], and SymDiff checks differential properties [9]. VeriBin decides at the binary level whether a patch is safe to apply by checking functional equivalence, on 86 samples [11]. All of these run beside the update. Ours is the same family of reasoning moved into the admission path of a running runtime, with a certificate the process re-checks, and extended with contracts, effect summaries and heap migration as obligations of the same certificate.

Certificates and load-time checking. Proof-carrying code ships a proof with native code and lets the consumer validate it [12]; the eBPF verifier checks untrusted kernel extensions before loading them [21]. Both verify a unit of code once. We verify a *transition* between two versions of a running program.

Translation validation. Pnueli et al. and Necula validate individual compilations rather than the compiler [14, 15]; Alive2 does so for LLVM IR [16]. Our relational verifier solves a different problem, comparing source versions of a function, but uses the same solver-based discipline, and the compiler's translation validation is the part of the trusted base that shrinks as it covers more transformations.

Automatically proposed patches. Repair systems and language models propose patches, and recent work asks how to judge them, for example by checking patch equivalence or by machine-checked proofs. **[TODO: do a real literature search of 2023 to 2026 work on checking automatically proposed patches and on verified hot patching, and fix this paragraph; the searches so far found VeriBin and agentic patch-equivalence work but this section has not been surveyed.]**

12 Conclusion

A running program should not have to take a patch on faith. By making the compiler's proofs about contracts, effects and types the vocabulary of an update, and by making the process the party that re-checks them, a live update can be refused for exactly the reasons a build would be refused. **[TODO: one paragraph of results once the evaluation exists.]** The definition, the checker and the protocol are specified here; the relational prover and the evaluation are the work that decides whether the approach covers realistic updates.

13 Artifact and reproducibility

The compiler, runtime, prover, corpus and harness are in one repository, built with a pinned Rust toolchain; the solver is Z3 at the version recorded in every certificate. **[TODO: repository location, commit, build and run instructions, the corpus manifest, and how to reproduce each table.]**

References

1. Michael Hicks and Jonathan T. Moore and Scott Nettles. Dynamic Software Updating. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2001.
2. Iulian Neamtiu and Michael Hicks and Gareth Stoyle and Manuel Oriol. Practical Dynamic Software Updating for C. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2006.
3. Christopher M. Hayden and Edward K. Smith and Michail Denchev and Michael Hicks and Jeffrey S. Foster. Kitsune: Efficient, General-purpose Dynamic Software Updating for C. *Proceedings of the ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2012.
4. Jeff Arnold and M. Frans Kaashoek. Ksplice: Automatic Rebootless Kernel Updates. *Proceedings of the ACM European Conference on Computer Systems (EuroSys)*, 2009.
5. Lu{\'i}s Pina and Lu{\'i}s Veiga and Michael Hicks. Rubah: DSU for Java on a Stock JVM. *Proceedings of the ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2014. [check authors: title and venue verified by web search, author list from memory]
6. Gareth Stoyle and Michael Hicks and Gavin Bierman and Peter Sewell and Iulian Neamtiu. Mutatis Mutandis: Safe and Predictable Dynamic Software Updating. *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2005.
7. Linux kernel documentation. Livepatch. *docs.kernel.org/livepatch/livepatch.html*, 2026.
8. Benny Godlin and Ofer Strichman. Regression Verification: Proving the Equivalence of Similar Programs. *Software Testing, Verification and Reliability*, 2013. [check venue and year: title and authors verified by web search]
9. Shuvendu K. Lahiri and Chris Hawblitzel and Ming Kawaguchi and Henrique Reb{\^e}lo. SymDiff: A Language-Agnostic Semantic Diff Tool for Imperative Programs. *Computer Aided Verification (CAV)*, 2012. [check: from memory; a search confirmed SymDiff exists as a differential verification tool]
10. Christopher M. Hayden and Stephen Magill and Michael Hicks and Nate Foster and Jeffrey S. Foster. Specifying and Verifying the Correctness of Dynamic Software Updates. *Verified Software: Theories, Tools, Experiments (VSTTE)*, 2012.
11. Hongwei Wu and Jianliang Wu and Ruoyu Wu and Ayushi Sharma and Aravind Machiry and Antonio Bianchi. VeriBin: Adaptive Verification of Patches at the Binary Level. *Network and Distributed System Security Symposium (NDSS)*, 2025.
12. George C. Necula and Peter Lee. Safe Kernel Extensions Without Run-Time Checking. *Proceedings of the Symposium on Operating Systems Design and Implementation (OSDI)*, 1996.

13. Leonardo de Moura and Nikolaj Bj{\o}rner. Z3: An Efficient SMT Solver. *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008. [check: from memory, not looked up]
14. Amir Pnueli and Michael Siegel and Eli Singerman. Translation Validation. *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 1998. [check: cited from the codira_tv module documentation, not looked up]
15. George C. Necula. Translation Validation for an Optimizing Compiler. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2000. [check: cited from the codira_tv module documentation, not looked up]
16. Nuno P. Lopes and Juneyoung Lee and Chung-Kil Hur and Zhengyang Liu and John Regehr. Alive2: Bounded Translation Validation for LLVM. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2021.
17. Nick Benton. Simple Relational Correctness Proofs for Static Analyses and Program Transformations. *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2004. [check: from memory, not looked up]
18. Gilles Barthe and Juan Manuel Crespo and C{\'e}sar Kunz. Relational Verification Using Product Programs. *Formal Methods (FM)*, 2011. [check: from memory, not looked up]
19. Joe Armstrong. Making Reliable Distributed Systems in the Presence of Software Errors. *PhD thesis, Royal Institute of Technology, Stockholm*, 2003. [check: from memory, not looked up]
20. authors listed at arXiv:2203.12132. Runtime Software Patching: Taxonomy, Survey and Future Directions. *arXiv:2203.12132*, 2022. [check authors: exists, found by web search; author list not recorded]
21. Elazar Gershuni and others. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2019.